

Git & GitHub Command Cheat Sheet for Cloud Engineering

1. Basic Git Setup

Check Git version:

```
git --version
```

Set your username:

```
git config --global user.name "Waliur Sun"
```

Set your email:

```
git config --global user.email "your-email@example.com"
```

Check Git configuration:

```
git config --list
```

Set default branch name to **main**:

```
git config --global init.defaultBranch main
```

2. Create a New Git Repository

Create a project folder:

```
mkdir cloud-security-project  
cd cloud-security-project
```

Initialize Git:

```
git init
```

Check status:

```
git status
```

Add all files:

```
git add .
```

Commit:

```
git commit -m "Initial project setup"
```

3. Clone an Existing GitHub Repository

Clone using HTTPS:

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

Go into the repo:

```
cd REPOSITORY
```

Example:

```
git clone  
https://github.com/waliurrahmansun003/linux-cloudops-command-  
center.git  
cd linux-cloudops-command-center
```

4. Check Repository Status

Check modified/untracked files:

```
git status
```

Short status:

```
git status -s
```

Show file changes:

```
git diff
```

Show staged changes:

```
git diff --staged
```

5. Add and Commit Files

Add one file:

```
git add README.md
```

Add one folder:

```
git add scripts/
```

Add everything:

```
git add .
```

Commit:

```
git commit -m "Add Linux health check script"
```

Good cloud project commit examples:

```
git commit -m "Add Terraform S3 bucket configuration"
```

```
git commit -m "Add IAM least privilege policy"
```

```
git commit -m "Add CloudTrail logging module"
```

```
git commit -m "Add Dockerfile for Flask app"
git commit -m "Add Kubernetes deployment manifest"
git commit -m "Add GitHub Actions CI pipeline"
git commit -m "Fix security group inbound rules"
git commit -m "Update README with architecture diagram"
```

6. View Commit History

View commit history:

```
git log
```

Compact commit history:

```
git log --oneline
```

Graph view:

```
git log --oneline --graph --all
```

Detailed recent commits:

```
git log -5
```

Show who changed each line:

```
git blame README.md
```

Show a specific commit:

```
git show COMMIT_ID
```

7. Connect Local Repo to GitHub

Create a GitHub repo first, then connect:

```
git remote add origin  
https://github.com/USERNAME/REPOSITORY.git
```

Check remote:

```
git remote -v
```

Rename branch to main:

```
git branch -M main
```

Push first time:

```
git push -u origin main
```

After first push:

```
git push
```

8. Pull Latest Changes

Pull latest changes from GitHub:

```
git pull
```

Pull from main branch:

```
git pull origin main
```

Fetch without merging:

```
git fetch
```

See remote branches:

```
git branch -r
```

9. Branching Commands

List local branches:

```
git branch
```

List all branches:

```
git branch -a
```

Create new branch:

```
git branch feature/add-cloudtrail-module
```

Switch to branch:

```
git checkout feature/add-cloudtrail-module
```

Create and switch in one command:

```
git checkout -b feature/add-cloudtrail-module
```

Modern command:

```
git switch -c feature/add-cloudtrail-module
```

Switch back to main:

```
git checkout main
```

or:

```
git switch main
```


Delete local branch:

```
git branch -d feature/add-cloudtrail-module
```

Force delete local branch:

```
git branch -D feature/add-cloudtrail-module
```

10. Cloud Engineering Branch Naming Examples

Use professional branch names:

feature/add-s3-encryption

feature/add-iam-policy

feature/add-cloudtrail-logging

feature/add-guardduty-detector

feature/add-terraform-backend

feature/add-dockerfile

feature/add-k8s-deployment

feature/add-github-actions-ci

fix/security-group-rules

fix/terraform-variable-error

fix/docker-build-failure

docs/update-readme

docs/add-architecture-diagram

security/remove-hardcoded-secret

security/add-kms-encryption

hotfix/block-public-s3-access

11. Merge Branches

Switch to main:

```
git checkout main
```

Pull latest main:

```
git pull origin main
```

Merge feature branch:

```
git merge feature/add-cloudtrail-module
```

Push updated main:

```
git push origin main
```

12. Merge Conflict Commands

When conflict happens, check status:

```
git status
```

Open conflicted file:

```
nano FILE_NAME
```

Conflict will look like this:

```
<<<<<< HEAD
bucket = "secure-logs-prod"
=====
bucket = "secure-logs-dev"
>>>>>> feature/update-bucket-name
```

Fix the file manually, then:

```
git add FILE_NAME
git commit -m "Resolve merge conflict in Terraform bucket
configuration"
```

Abort merge if needed:

```
git merge --abort
```

13. Rebase Basics

Rebase feature branch onto latest main:

```
git checkout feature/add-cloudtrail-module
git fetch origin
```

```
git rebase origin/main
```

Continue after fixing conflict:

```
git add .  
git rebase --continue
```

Abort rebase:

```
git rebase --abort
```

Use carefully. For beginner cloud projects, normal merge is usually safer.

14. Undo Changes Before Commit

Discard changes in one file:

```
git checkout -- FILE_NAME
```

Modern version:

```
git restore FILE_NAME
```

Discard all unstaged changes:

```
git restore .
```

Unstage a file:

```
git restore --staged FILE_NAME
```

Unstage everything:

```
git restore --staged .
```

15. Undo Commits Safely

Safe Undo: `git revert`

Use this when you already pushed or are working with a team.

```
git revert HEAD
```

Revert a specific commit:

```
git revert COMMIT_ID
```

Cloud example:

```
git revert abc1234
```

Commit message example:

Revert risky security group change

Dangerous Undo: `git reset`

Only use before pushing or in personal practice repos.

Go back one commit but keep changes:

```
git reset --soft HEAD~1
```

Go back one commit and unstage changes:

```
git reset --mixed HEAD~1
```

Delete last commit and changes:

```
git reset --hard HEAD~1
```

Warning:

```
git reset --hard
```

can permanently delete work.

16. Stash Temporary Work

Save current changes without committing:

git stash

Show stashes:

git stash list

Apply latest stash:

git stash apply

Apply and remove stash:

git stash pop

Stash with message:

git stash push -m "temporary terraform changes"

Delete all stashes:

git stash clear

Cloud use case:

You are editing Terraform files, but need to quickly switch branches to fix a production bug.

17. Tags and Releases

Create tag:

```
git tag v1.0.0
```

Create annotated tag:

```
git tag -a v1.0.0 -m "Initial stable release"
```

List tags:

```
git tag
```

Show tag:

```
git show v1.0.0
```

Push one tag:

```
git push origin v1.0.0
```

Push all tags:

```
git push --tags
```

Delete local tag:


```
git tag -d v1.0.0
```

Delete remote tag:

```
git push origin --delete v1.0.0
```

Cloud release examples:

```
v1.0.0-linux-cloudops
```

```
v1.1.0-add-cloudtrail
```

```
v1.2.0-add-guardduty
```

```
v2.0.0-production-ready
```

18. **.gitignore** for Cloud Projects

Create **.gitignore**:

```
touch .gitignore
```

Recommended cloud project **.gitignore**:

```
# OS files
```

```
.DS_Store
```

```
Thumbs.db
```

```
# Logs
```

```
*.log
```

Environment variables

- .env
- .env.local
- .env.production

SSH keys and certificates

- *.pem
- *.key
- *.crt

Terraform

- .terraform/
- *.tfstate
- *.tfstate.backup
- .terraform.lock.hcl
- crash.log
- override.tf
- override.tf.json
- *_override.tf
- *_override.tf.json

AWS credentials

- .aws/
- credentials
- config

Python

- __pycache__/
- *.pyc

venv/
.env/

Node
node_modules/

IDE
.vscode/
.idea/

Docker
docker-compose.override.yml

Very important: never commit these:

.env
*.pem
*.key
*.tfstate
AWS credentials
database passwords
API keys

19. GitHub Pull Request Workflow

Create branch:

```
git checkout -b feature/add-terraform-backend
```

Make changes:

```
nano backend.tf
```

Add and commit:

```
git add .
```

```
git commit -m "Add Terraform remote backend configuration"
```

Push branch:

```
git push -u origin feature/add-terraform-backend
```

Then go to GitHub and open a Pull Request.

Good PR title:

Add Terraform remote backend configuration

Good PR description:

Summary

Added Terraform backend configuration for remote state management.

Changes

- Added S3 backend block

- Added DynamoDB state locking reference
- Updated README with backend setup steps

Security Considerations

- Terraform state files are excluded from Git
- Sensitive values are not committed

Testing

- Ran terraform fmt
- Reviewed git diff

Merge the PR after review.

20. GitHub Issues

Create GitHub issues for your project tasks.

Example issue titles:

Add CloudTrail module
Add GuardDuty detector
Add Terraform backend
Add README architecture diagram
Fix security group overly permissive rule
Add CI pipeline for Terraform validation
Add Trivy scan for Docker image
Add screenshots for project evidence

Good issue format:

Task

Add GuardDuty detector to the Terraform project.

Why

GuardDuty provides threat detection for AWS accounts.

Acceptance Criteria

- GuardDuty detector is created
- Terraform code is formatted
- README is updated
- Screenshot is added

21. GitHub Actions Basics for Cloud Projects

Create workflow folder:

```
mkdir -p .github/workflows
```

Create workflow file:

```
nano .github/workflows/ci.yml
```

Basic shell script CI:

```
name: Basic CI
```

on:

push:

branches: ["main"]

pull_request:

jobs:

validate:

runs-on: ubuntu-latest

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Check shell scripts

run: |

find scripts -name "*.sh" -type f -print

bash -n scripts/*.sh

Commit:

git add .github/workflows/ci.yml

git commit -m "Add GitHub Actions CI workflow"

git push

22. Terraform Git Workflow

Before committing Terraform code:

```
terraform fmt
terraform validate
terraform plan
```

Git commands:

```
git checkout -b feature/add-vpc-module
git add .
git commit -m "Add Terraform VPC module"
git push -u origin feature/add-vpc-module
```

Good Terraform commit examples:

```
git commit -m "Add VPC module with public and private subnets"
git commit -m "Add S3 bucket encryption with KMS"
git commit -m "Add IAM least privilege policy"
git commit -m "Add CloudTrail multi-region logging"
git commit -m "Add GuardDuty threat detection"
git commit -m "Fix Terraform variable validation"
```

23. Docker Git Workflow

Before committing Docker code:

```
docker build -t app-test .
docker run -p 8080:8080 app-test
```


Git commands:

```
git checkout -b feature/add-dockerfile
git add Dockerfile
git commit -m "Add Dockerfile for application container"
git push -u origin feature/add-dockerfile
```

Good Docker commit examples:

```
git commit -m "Add non-root user to Dockerfile"
git commit -m "Reduce Docker image size"
git commit -m "Add Trivy scan workflow"
git commit -m "Fix container port mapping"
```

24. Kubernetes Git Workflow

Before committing Kubernetes YAML:

```
kubectl apply --dry-run=client -f deployment.yaml
```

Git commands:

```
git checkout -b feature/add-k8s-manifests
git add k8s/
git commit -m "Add Kubernetes deployment and service manifests"
git push -u origin feature/add-k8s-manifests
```

Good Kubernetes commit examples:

```
git commit -m "Add Kubernetes deployment manifest"  
git commit -m "Add service and ingress configuration"  
git commit -m "Add resource limits for application pods"  
git commit -m "Add network policy for pod isolation"  
git commit -m "Fix container image tag"
```

25. Security Commands for Git

Search for secrets before committing:

```
git diff
```

Search for AWS keys:

```
grep -r "AKIA" .
```

Search for passwords:

```
grep -ri "password" .
```

Search for secrets:

```
grep -ri "secret" .
```

Remove file from Git tracking but keep locally:

```
git rm --cached .env
```

Commit removal:

```
git commit -m "Remove environment file from tracking"
```

If secret was committed, rotate it immediately. Do not just delete it from GitHub.

26. Useful GitHub CLI Commands

Install GitHub CLI first if needed.

Login:

```
gh auth login
```

Create repo:

```
gh repo create gitops-version-control-lab --public --source=.  
--remote=origin --push
```

View repo:

```
gh repo view --web
```

Create issue:

```
gh issue create --title "Add Terraform validation workflow" --body  
"Add GitHub Actions workflow to run terraform fmt and validate."
```

List issues:

```
gh issue list
```

Create pull request:

```
gh pr create --title "Add Terraform backend" --body "Adds  
Terraform backend configuration and documentation."
```

List pull requests:

```
gh pr list
```

Checkout PR locally:

```
gh pr checkout PR_NUMBER
```

Merge PR:

```
gh pr merge PR_NUMBER
```

27. Professional Daily Git Workflow

Use this workflow for your cloud projects:

```
git checkout main  
git pull origin main  
git checkout -b feature/add-new-cloud-security-control
```

Make changes.

Check changes:

```
git status  
git diff
```

Add and commit:

```
git add .  
git commit -m "Add new cloud security control"
```

Push:

```
git push -u origin feature/add-new-cloud-security-control
```

Open Pull Request on GitHub.

After merge:

```
git checkout main  
git pull origin main  
git branch -d feature/add-new-cloud-security-control
```

28. Most Important Commands to Memorize First

git init
git clone
git status
git add .
git commit -m "message"
git log --oneline
git diff
git branch
git checkout -b branch-name
git checkout main
git merge branch-name
git pull
git push
git remote -v
git remote add origin URL
git restore FILE
git restore --staged FILE
git revert HEAD
git stash
git stash pop
git tag -a v1.0.0 -m "message"

29. Cloud Engineer Git Mindset

For cloud projects, Git is not just for code. Git is used for:

- Terraform infrastructure
- Kubernetes manifests
- Dockerfiles
- CI/CD workflows
- Bash automation scripts
- Security policies
- IAM policy documents
- Documentation
- Architecture diagrams
- Incident response runbooks
- Change history

A cloud engineer should be able to explain:

I created a feature branch, made infrastructure changes, reviewed the diff, committed with a clear message, opened a pull request, resolved conflicts, merged safely, tagged a release, and used revert to roll back risky changes.

30. Best Git Commands for Your Resume-Level Project

Use these in your GitOps project and screenshot them:

```
git log --oneline --graph --all
```

git status
git diff
git branch -a
git merge feature/add-security-tags
git merge --abort
git revert HEAD
git tag -a v1.0.0 -m "Initial stable release"
git show v1.0.0
git remote -v

These prove you understand real version control workflows, not just basic Git.